

Assessment 2: Machine learning and real-time streaming

Stefan Garevski (33759839)

Dataset reuse and train/stream split

```
#initial imports
from pyspark.sql import SparkSession
from pyspark.sql.types import *
from pyspark.sql.functions import (
    col,
    when,
    hour,
    to_timestamp,
    countDistinct,
    sum,
    round,
    substring,
    row_number,
    spark_partition_id,
    min,
    max,
    avg,
    count
)
from pyspark.sql.window import Window
from pyspark.ml.feature import Imputer
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

from pyspark.ml.classification import RandomForestClassifier,
LogisticRegression
from pyspark.ml import Pipeline
from pyspark.ml import PipelineModel

from pyspark.ml.feature import (
    StringIndexer,
    OneHotEncoder,
    VectorAssembler
)

from pyspark.sql import functions as F
#initiate spark session
spark = SparkSession.builder \
    .appName("Victorian Road Crash Analysis") \
    .getOrCreate()
```

```
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-
defaults.properties
26/10/04 10:57:04 WARN Utils: Your hostname, Stefans-MacBook-
Air.local, resolves to a loopback address: 127.0.0.1; using
192.168.1.125 instead (on interface en0)
26/10/04 10:57:04 WARN Utils: Set SPARK_LOCAL_IP if you need to bind
to another address
:: loading settings :: url =
jar:file:/opt/anaconda3/envs/IT05202/lib/python3.13/site-packages/
pyspark/jars/ivy-2.5.3.jar!/org/apache/ivy/core/settings/
ivysettings.xml
Ivy Default Cache set to: /Users/stefangarevski/.ivy2.5.2/cache
The jars for the packages stored in:
/Users/stefangarevski/.ivy2.5.2/jars
org.apache.spark#spark-sql-kafka-0-10_2.13 added as a dependency
:: resolving dependencies :: org.apache.spark#spark-submit-parent-
19d2beb9-9caf-415e-b035-69db7dc00734;1.0
   confs: [default]
   found org.apache.spark#spark-sql-kafka-0-10_2.13;4.2.0 in central
   found org.apache.spark#spark-token-provider-kafka-0-10_2.13;4.2.0
in central
   found org.apache.kafka#kafka-clients;3.9.2 in central
   found at.yawk.lz4#lz4-java;1.11.0 in central
   found org.xerial.snappy#snappy-java;1.1.10.8 in central
   found org.slf4j#slf4j-api;2.0.17 in central
   found org.apache.hadoop#hadoop-client-runtime;3.5.0 in central
   found org.apache.hadoop#hadoop-client-api;3.5.0 in central
   found com.google.code.findbugs#jsr305;3.0.0 in central
   found org.scala-lang.modules#scala-parallel-
collections_2.13;1.2.0 in central
   found org.apache.commons#commons-pool2;2.13.1 in central
:: resolution report :: resolve 148ms :: artifacts dl 4ms
:: modules in use:
at.yawk.lz4#lz4-java;1.11.0 from central in [default]
com.google.code.findbugs#jsr305;3.0.0 from central in [default]
org.apache.commons#commons-pool2;2.13.1 from central in [default]
org.apache.hadoop#hadoop-client-api;3.5.0 from central in
[default]
org.apache.hadoop#hadoop-client-runtime;3.5.0 from central in
[default]
org.apache.kafka#kafka-clients;3.9.2 from central in [default]
org.apache.spark#spark-sql-kafka-0-10_2.13;4.2.0 from central in
[default]
org.apache.spark#spark-token-provider-kafka-0-10_2.13;4.2.0 from
central in [default]
org.scala-lang.modules#scala-parallel-collections_2.13;1.2.0 from
central in [default]
org.slf4j#slf4j-api;2.0.17 from central in [default]
org.xerial.snappy#snappy-java;1.1.10.8 from central in [default]
```

-----		modules						artifacts	
	conf	number	search	dwnlded	evicted			number	
dwnlded									

	default	11	0	0	0			11	0

```

:: retrieving :: org.apache.spark#spark-submit-parent-19d2beb9-9caf-
415e-b035-69db7dc00734
  confs: [default]
    0 artifacts copied, 11 already retrieved (0kB/3ms)
26/10/04 10:57:05 WARN NativeCodeLoader: Unable to load native-hadoop
library for your platform... using builtin-java classes where
applicable
Using Spark's default log4j profile: org/apache/spark/log4j2-
defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use
setLogLevel(newLevel).
/opt/anaconda3/envs/IT05202/lib/python3.13/site-packages/pyspark/testi
ng/utils.py:127: FutureWarning: PySpark does not yet fully support
pandas >= 3.0.0. Some features may not work correctly. It is
recommended to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()

```

#define schema of variables in dataset

```

schema = StructType([
    StructField("ACCIDENT_NO", StringType(), True),
    StructField("ACCIDENT_DATE", StringType(), True),
    StructField("ACCIDENT_TIME", StringType(), True),
    StructField("ACCIDENT_TYPE", StringType(), True),
    StructField("DAY_OF_WEEK", StringType(), True),
    StructField("DCA_CODE", StringType(), True),
    StructField("DCA_CODE_DESCRIPTION", StringType(), True),
    StructField("LIGHT_CONDITION", StringType(), True),
    StructField("POLICE_ATTEND", StringType(), True),
    StructField("ROAD_GEOMETRY", StringType(), True),
    StructField("SEVERITY", StringType(), True),
    StructField("SPEED_ZONE", StringType(), True),
    StructField("RUN_OFFROAD", StringType(), True),
    StructField("ROAD_NAME", StringType(), True),
    StructField("ROAD_TYPE", StringType(), True),
    StructField("ROAD_ROUTE_1", StringType(), True),
    StructField("LGA_NAME", StringType(), True),
    StructField("DTP_REGION", StringType(), True),

```

```

StructField("LATITUDE", DoubleType(), True),
StructField("LONGITUDE", DoubleType(), True),
StructField("VICGRID_X", DoubleType(), True),
StructField("VICGRID_Y", DoubleType(), True),
StructField("TOTAL_PERSONS", IntegerType(), True),
StructField("INJ_OR_FATAL", IntegerType(), True),
StructField("FATALITY", IntegerType(), True),
StructField("SERIOUSINJURY", IntegerType(), True),
StructField("OTHERINJURY", IntegerType(), True),
StructField("NONINJURED", IntegerType(), True),
StructField("MALES", IntegerType(), True),
StructField("FEMALES", IntegerType(), True),
StructField("BICYCLIST", IntegerType(), True),
StructField("PASSENGER", IntegerType(), True),
StructField("DRIVER", IntegerType(), True),
StructField("PEDESTRIAN", IntegerType(), True),
StructField("PILLION", IntegerType(), True),
StructField("MOTORCYCLIST", IntegerType(), True),
StructField("UNKNOWN", IntegerType(), True),
StructField("PED_CYCLIST_5_12", IntegerType(), True),
StructField("PED_CYCLIST_13_18", IntegerType(), True),
StructField("OLD_PED_65_AND_OVER", IntegerType(), True),
StructField("OLD_DRIVER_75_AND_OVER", IntegerType(), True),
StructField("YOUNG_DRIVER_18_25", IntegerType(), True),
StructField("NO_OF_VEHICLES", IntegerType(), True),
StructField("HEAVYVEHICLE", IntegerType(), True),
StructField("PASSENGERVEHICLE", IntegerType(), True),
StructField("MOTORCYCLE", IntegerType(), True),
StructField("PT_VEHICLE", IntegerType(), True),
StructField("DEG_URBAN_NAME", StringType(), True),
StructField("SRNS", StringType(), True),
StructField("RMA", StringType(), True),
StructField("DIVIDED", StringType(), True),
StructField("STAT_DIV_NAME", StringType(), True)
])

#load in csv data
file_path = "../Assessment 1/vic_road_crash_data.csv"

crashes = spark.read \
    .option("header", True) \
    .schema(schema) \
    .csv(file_path)

# create split
train_df, stream_df = crashes.randomSplit([0.7, 0.3], seed=42)
total = crashes.count()

# check percentage split
print("Training:", train_df.count(),

```

```
f"({train_df.count()/total:.2%})")
print("Streaming:", stream_df.count(),
f"({stream_df.count()/total:.2%})")

# save as parquet
stream_df.write.mode("overwrite").parquet("data/stream_data.parquet")

26/10/04 10:57:08 WARN SparkStringUtils: Truncated the string
representation of a plan since it was too large. This behavior can be
adjusted by setting 'spark.sql.debug.maxToStringFields'.

Training: 140622 (70.19%)
Streaming: 59730 (29.81%)

26/10/04 10:57:11 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 95.00% for 8 writers
26/10/04 10:57:11 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 84.44% for 9 writers
26/10/04 10:57:11 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 76.00% for 10 writers
26/10/04 10:57:11 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 84.44% for 9 writers
26/10/04 10:57:11 WARN MemoryManager: Total allocation exceeds 95.00%
(1,020,054,720 bytes) of heap memory
Scaling row group sizes to 95.00% for 8 writers
```

Part A: The ML pipeline (45%)

1. ML task selection

For Part A, I have selected classification as the machine learning task. The rationale is that the VicRoads crash dataset contains an appropriate categorical target variable called **SEVERITY**, which represents the severity category of each recorded road crash.

The dataset contains a mix of categorical and numerical variables that can be used as features in the model. Categorical variables include **ACCIDENT_TYPE**, **DAY_OF_WEEK**, **LIGHT_CONDITION**, **ROAD_GEOMETRY**, **ROAD_TYPE**, **SPEED_ZONE**, **RUN_OFFROAD**, and **LGA_NAME**. Numerical variables include **TOTAL_PERSONS**, **INJ_OR_FATAL**, **FATALITY**, **SERIOUSINJURY**, **OTHERINJURY**, **NO_OF_VEHICLES**, etc. These variables describe the crash, road environment, location, vehicles, and people involved. Therefore these variables can provide meaningful information for predicting the crash severity.

Classification is the most suitable in this instance because the objective is to predict which severity category a crash belongs to rather than predict a continuous numerical quantity. The **SEVERITY** column provides an existing categorical label that can be used, so there is no requirement to set any type of threshold for this activity.

Something like a regression is less suitable because most of the numerical variables generally describe counts or measurements associated with a crash rather than a prediction whose output would be a continuous number. Also, the **SEVERITY** variable is categorical rather than continuous.

Clustering is also not really suitable because it is primarily intended for situations where there may be no suitable target variable and the goal is to discover previously unknown groups or patterns in the data. In this instance, classification provides a more direct way to use the Victorian crash dataset.

Therefore, I will be using classification with **SEVERITY** as the target variable and relevant crash, road, vehicle, location, and environmental attributes as predictive features.

2. Feature engineering

```
# define potential categorical variables
```

```
pot_categorical_cols = [  
    "ACCIDENT_TYPE",  
    "DAY_OF_WEEK",  
    "LIGHT_CONDITION",  
    "POLICE_ATTEND",  
    "ROAD_GEOMETRY",  
    "SPEED_ZONE",  
    "RUN_OFFROAD",  
    "ROAD_NAME",  
    "ROAD_TYPE",  
    "LGA_NAME",  
    "DTP_REGION",  
    "DEG_URBAN_NAME",  
    "SRNS",  
    "RMA",  
    "DIVIDED",  
    "STAT_DIV_NAME",  
    "DCA_CODE",  
    "DCA_CODE_DESCRIPTION"  
]
```

```
#define potential numerical variables
```

```
pot_numeric_cols = [  
    "LATITUDE",  
    "LONGITUDE",  
    "TOTAL_PERSONS",  
    "NO_OF_VEHICLES",  
    "MALES",  
    "FEMALES",  
]
```

```

    "BICYCLIST",
    "PASSENGER",
    "DRIVER",
    "PEDESTRIAN",
    "PILLION",
    "MOTORCYCLIST",
    "UNKNOWN",
    "PED_CYCLIST_5_12",
    "PED_CYCLIST_13_18",
    "OLD_PED_65_AND_OVER",
    "OLD_DRIVER_75_AND_OVER",
    "HEAVYVEHICLE",
    "YOUNG_DRIVER_18_25",
    "PASSENGERVEHICLE",
    "MOTORCYCLE",
    "PT_VEHICLE"
]

#check categorical feature cardinality
from pyspark.sql import functions as F

for col in pot_categorical_cols:
    stats = crashes.select(
        F.countDistinct(col).alias("distinct"),
        F.count("*").alias("total")
    ).collect()[0]

    print(
        f"{col:25} "
        f"distinct={stats['distinct']:5} "
        f"total={stats['total']}"
    )

```

ACCIDENT_TYPE	distinct=	9	total=200352
DAY_OF_WEEK	distinct=	7	total=200352
LIGHT_CONDITION	distinct=	7	total=200352
POLICE_ATTEND	distinct=	3	total=200352
ROAD_GEOMETRY	distinct=	9	total=200352
SPEED_ZONE	distinct=	13	total=200352
RUN_OFFROAD	distinct=	2	total=200352
ROAD_NAME	distinct=	14558	total=200352
ROAD_TYPE	distinct=	80	total=200352
LGA_NAME	distinct=	87	total=200352
DTP_REGION	distinct=	8	total=200352
DEG_URBAN_NAME	distinct=	7	total=200352
SRNS	distinct=	4	total=200352
RMA	distinct=	6	total=200352
DIVIDED	distinct=	2	total=200352
STAT_DIV_NAME	distinct=	2	total=200352

DCA_CODE	distinct=	81	total=200352
DCA_CODE_DESCRIPTION	distinct=	81	total=200352

#check numerical variable variance

```
for col in pot_numeric_cols:
    stats = crashes.select(
        F.countDistinct(col).alias("distinct"),
        F.min(col).alias("min"),
        F.max(col).alias("max"),
        F.avg(col).alias("mean")
    ).collect()[0]

    print(
        f"{col:25} "
        f"distinct={stats['distinct']:5} "
        f"min={stats['min']} "
        f"max={stats['max']} "
        f"mean={stats['mean']:.2f}"
    )
```

LATITUDE	distinct=106932	min=-39.03083	max=-34.115696	
mean=-37.72				
LONGITUDE	distinct=79276	min=140.96648	max=149.75746	
mean=144.97				
TOTAL_PERSONS	distinct=	45	min=1	max=97
NO_OF_VEHICLES	distinct=	16	min=1	max=21
MALES	distinct=	26	min=0	max=46
FEMALES	distinct=	30	min=0	max=51
BICYCLIST	distinct=	9	min=0	max=8
PASSENGER	distinct=	42	min=0	max=95
DRIVER	distinct=	16	min=0	max=21
PEDESTRIAN	distinct=	10	min=0	max=11
PILLION	distinct=	3	min=0	max=2
MOTORCYCLIST	distinct=	9	min=0	max=9
UNKNOWN	distinct=	24	min=0	max=52
PED_CYCLIST_5_12	distinct=	6	min=0	max=8
PED_CYCLIST_13_18	distinct=	5	min=0	max=5
OLD_PED_65_AND_OVER	distinct=	4	min=0	max=3
OLD_DRIVER_75_AND_OVER	distinct=	4	min=0	max=3
HEAVYVEHICLE	distinct=	5	min=0	max=4
YOUNG_DRIVER_18_25	distinct=	6	min=0	max=5
PASSENGERVEHICLE	distinct=	16	min=0	max=19
MOTORCYCLE	distinct=	9	min=0	max=9
PT_VEHICLE	distinct=	3	min=0	max=2

define categorical variables

```
categorical_cols = [
    "ACCIDENT_TYPE",
    "DAY_OF_WEEK",
    "LIGHT_CONDITION",
```

```

    "POLICE_ATTEND",
    "ROAD_GEOMETRY",
    "SPEED_ZONE",
    "RUN_OFFROAD",
    "DTP_REGION",
    "DEG_URBAN_NAME",
    "DIVIDED",
    "STAT_DIV_NAME"
]

#define numerical variables
numeric_cols = [
    "TOTAL_PERSONS",
    "NO_OF_VEHICLES",
    "MALES",
    "FEMALES",
    "BICYCLIST",
    "PASSENGER",
    "DRIVER",
    "PEDESTRIAN",
    "PILLION",
    "MOTORCYCLIST",
    "UNKNOWN",
    "PED_CYCLIST_5_12",
    "PED_CYCLIST_13_18",
    "OLD_PED_65_AND_OVER",
    "OLD_DRIVER_75_AND_OVER",
    "HEAVYVEHICLE",
    "YOUNG_DRIVER_18_25",
    "PASSENGERVEHICLE",
    "MOTORCYCLE",
    "PT_VEHICLE"
]

# check for variables that have null values
for col in numeric_cols:
    stats = crashes.select(
        F.sum(F.when(F.col(col).isNull(),
1).otherwise(0)).alias("nulls"),
        F.sum(F.when(F.isnan(F.col(col)),
1).otherwise(0)).alias("NaNs")
    ).collect()[0]

    if stats["nulls"] > 0 or stats["NaNs"] > 0:
        print(
            f"{col:25} "
            f"nulls={stats['nulls']} "
            f"NaNs={stats['NaNs']}"
        )

```

NO_OF_VEHICLES	nulls=9 NaNs=0
HEAVYVEHICLE	nulls=9 NaNs=0
PASSENGERVEHICLE	nulls=9 NaNs=0
MOTORCYCLE	nulls=9 NaNs=0
PT_VEHICLE	nulls=9 NaNs=0

```

# convert categorical variables to numerical indices
indexers = [
    StringIndexer(
        inputCol=col,
        outputCol=col + "_index",
        handleInvalid="keep"
    )
    for col in categorical_cols
]

# one hot encode for categorical variables
encoder = OneHotEncoder(
    inputCols=[col + "_index" for col in categorical_cols],
    outputCols=[col + "_encoded" for col in categorical_cols]
)

# set the target variable
label_indexer = StringIndexer(
    inputCol="SEVERITY",
    outputCol="label",
    handleInvalid="keep"
)

# use imputer for variables that have missing values
imputed_numeric_cols = [
    f"{col}_imputed" for col in numeric_cols
]

#use median for imputation
imputer = Imputer(
    inputCols=numeric_cols,
    outputCols=imputed_numeric_cols,
    strategy="median"
)

# combine categorical and numerical variables into single feature vector
assembler = VectorAssembler(
    inputCols=(
        [f"{col}_encoded" for col in categorical_cols]
        + imputed_numeric_cols
    ),
    outputCol="features",
    handleInvalid="keep"

```

```

)

# create the preprocessing pipeline
preprocessing_pipeline = Pipeline(
    stages=indexers + [encoder, label_indexer, imputer, assembler]
)

# complete train/eval split
train_data, test_data = train_df.randomSplit([0.8, 0.2], seed=42)

# fit preprocessing pipeline using training data only
preprocessor_model = preprocessing_pipeline.fit(train_data)

# transform the training data
train_prepared = preprocessor_model.transform(train_data)

# transform the unseen test data using the fitted preprocessing
pipeline
test_prepared = preprocessor_model.transform(test_data)

```

Documentation

Feature selection was performed with three objectives; prevent target leakage, retain variables with meaningful predictive information, and remove features demonstrating high-cardinality or low information.

SEVERITY is the classification target and is therefore excluded from the list of variables for modelling. It is transformed separately using **StringIndexer** to produce the label column required by spark.

The variables **FATALITY**, **SERIOUSINJURY**, **OTHERINJURY**, **INJ_OR_FATAL**, and **NONINJURED** are excluded because they describe injury and fatality outcomes resulting from the crash. These variables are closely related to the crash severity outcome and could allow the model to infer the target using post-crash information rather than learning from the circumstances associated with the crash. Excluding these variables reduces the risk of target leakage.

ACCIDENT_NO is excluded because it is an identifier rather than providing any meaningful information. **DCA_CODE** and **DCA_CODE_DESCRIPTION** are also excluded because they provide detailed crash classifications that may encode information closely associated with the crash outcome.

The selected categorical describe crash type, temporal conditions, lighting, road characteristics, speed environment, road configuration, and geographic information. They are retained because these characteristics may provide useful predictive information.

Several categorical variables were not included in the final feature vector. For example, **ROAD_NAME** and **LGA_NAME** were excluded because they can contain many distinct categories. One-hot encoding very high-cardinality variables can substantially increase the dimensionality

of the feature vector, increase computational cost, and potentially encourage the model to learn location-specific patterns rather than general relationships. The broader geographic variables `DTP_REGION`, `DEG_URBAN_NAME`, and `STAT_DIV_NAME` were retained to provide geographic context at a more manageable level of granularity.

The selected numerical variables describe the number and composition of people and vehicles involved in each crash. These include `TOTAL_PERSONS`, `NO_OF_VEHICLES`, demographic counts, road-user counts, and vehicle-type counts.

`LATITUDE` and `LONGITUDE` were excluded from the final feature vector. Although they may contain predictive geographic information, excluding precise coordinates reduces the dependence of the model on exact location and keeps the feature set focused on crash, road, environmental, demographic, and vehicle characteristics.

Numerical features were also inspected for variation so that variables containing no meaningful variation could be identified and excluded. Variables with effectively zero variance would provide no discriminatory information to a classifier and would unnecessarily increase the feature vector.

The categorical variables are stored as strings and therefore cannot be directly supplied to the selected Spark ML classifier. `StringIndexer` converts each categorical value into a numerical category index. `OneHotEncoder` then represents these categories as binary indicator variables.

One-hot encoding is important because the numerical indices produced are arbitrary category identifiers and do not imply an ordinal relationship between categories.

The `handleInvalid="keep"` option is used so that previously unseen or invalid categorical values can be represented by an additional category rather than causing the transformation to fail. This is particularly important for the held-out streaming data, where a category may occur that was not present in the training subset.

Some numerical variables were found to have null values that need to be rectified. `Imputer` addresses this by locating the affected cells and inputting the median value across the dataset of that numerical variable. This is to ensure that there is a small chance of skew within the data.

`VectorAssembler` combines the one-hot encoded categorical variables and selected numerical variables into a single `features` vector. This is the input into spark ML classification algorithm.

The transformation pipeline is done in the following order:

1. `StringIndexer` transforms the categorical predictor variables
2. `OneHotEncoder` converts the indexed categorical variables into binary feature vectors
3. `StringIndexer` converts the `SEVERITY` target into the numerical `label` required by spark
4. `Imputer` converts empty cells into an imputed value, in this instance, the median value in the dataset
5. `VectorAssembler` combines the encoded categorical variables and numerical variables into the final `features` vector

3. Model comparison

4. Evaluation requirements

```
#generate Random Forest model
rf = RandomForestClassifier(
    labelCol="label",
    featuresCol="features",
    numTrees=100,
    seed=42
)

rf_pipeline = Pipeline(
    stages=indexers + [encoder, label_indexer, imputer, assembler, rf]
)

#train Random Forest
rf_model = rf_pipeline.fit(train_data)

#run predictions on the test data
rf_predictions = rf_model.transform(test_data)


#generate Logistic Regression model
lr = LogisticRegression(
    labelCol="label",
    featuresCol="features",
    maxIter=100,
    regParam=0.01
)

lr_pipeline = Pipeline(
    stages=indexers + [encoder, label_indexer, imputer, assembler, lr]
)

# train Logistic Regression
lr_model = lr_pipeline.fit(train_data)

#run predictions on the test data
lr_predictions = lr_model.transform(test_data)


#extract model metrics for both models
metrics = [
    "accuracy",
```

```

        "weightedPrecision",
        "weightedRecall",
        "f1"
    ]
    results = []
    for model_name, predictions in [
        ("Random Forest", rf_predictions),
        ("Logistic Regression", lr_predictions)
    ]:
        model_results = {"Model": model_name}
        for metric in metrics:
            evaluator = MulticlassClassificationEvaluator(
                labelCol="label",
                predictionCol="prediction",
                metricName=metric
            )
            model_results[metric] = evaluator.evaluate(predictions)
        results.append(model_results)
    results

[{'Model': 'Random Forest',
  'accuracy': 0.6424761018223479,
  'weightedPrecision': 0.625978071840842,
  'weightedRecall': 0.6424761018223479,
  'f1': 0.5452305439391208},
 {'Model': 'Logistic Regression',
  'accuracy': 0.6609143961906125,
  'weightedPrecision': 0.6399988142300648,
  'weightedRecall': 0.6609143961906125,
  'f1': 0.6327648305664093}]

```

General Metrics Comparison Table

```

#create comparison table
comparison_df = spark.createDataFrame(results)

# populate table with calculated metrics
comparison_df.select(
    "Model",
    F.round("accuracy", 4).alias("Accuracy"),
    F.round("weightedPrecision", 4).alias("Weighted Precision"),

```

```

    F.round("weightedRecall", 4).alias("Weighted Recall"),
    F.round("f1", 4).alias("Weighted F1")
).show(truncate=False)

+-----+-----+-----+-----+
+-----+
|Model|Accuracy|Weighted Precision|Weighted Recall|
Weighted F1|
+-----+-----+-----+-----+
+-----+
|Random Forest|0.6425|0.626|0.6425|
0.5452|
|Logistic Regression|0.6609|0.64|0.6609|
0.6328|
+-----+-----+-----+-----+
+-----+

```

generate confusion matrix for random forest

```
print("Random Forest confusion matrix:")
```

```
rf_predictions.groupBy(
    "label",
    "prediction"
).count().orderBy(
    "label",
    "prediction"
).show()
```

generate confusion matrix for logistic regression

```
print("Logistic Regression confusion matrix:")
```

```
lr_predictions.groupBy(
    "label",
    "prediction"
).count().orderBy(
    "label",
    "prediction"
).show()
```

Random Forest confusion matrix:

```

+-----+-----+-----+
|label|prediction|count|
+-----+-----+-----+
| 0.0|0.0|16930|
| 0.0|1.0|474|
| 1.0|0.0|9028|
| 1.0|1.0|1015|
| 2.0|0.0|349|
| 2.0|1.0|133|
| 3.0|0.0|2|
+-----+-----+-----+

```

Logistic Regression confusion matrix:

label	prediction	count
0.0	0.0	14892
0.0	1.0	2512
1.0	0.0	6475
1.0	1.0	3567
1.0	2.0	1
2.0	0.0	156
2.0	1.0	325
2.0	2.0	1
3.0	0.0	2

Final Model Selection Justification

Logistic regression achieved higher performance than random forest on all four evaluation metrics. Its accuracy was 0.6609, compared with 0.6425 for random forest, representing an improvement of approximately 1.84 percentage points. Logistic Regression also achieved higher weighted precision and weighted recall .

The largest difference occurred for weighted F1-score. logistic regression achieved a weighted F1-score of 0.6328, whereas random forest achieved 0.5452. This difference is important because weighted F1 provides a balance between precision and recall while accounting for the frequency of the severity classes.

The model selection also has to take into consideration the distribution of the target variable and the predictions of the model, not just considering accuracy. This is important because a classifier can obtain a relatively high accuracy by correctly predicting a more frequent class while performing less effectively on less frequent classes.

The confusion matrix provides additional support for this selection of logistic regression because it correctly classified substantially more observations from label 1 and produced fewer 1 for 0 misclassifications. However, neither model performs adequately on the rarer severity classes, particularly label 3. This limitation is noted.

Based on the observed results on the test data, logistic regression will be the final model. The higher weighted F1-score indicates that Logistic Regression provides a stronger balance between correctly identifying severity classes and limiting incorrect classifications under the evaluation conditions used in this experiment.

5. Model persistence

```
# save best model
bestModel = lr_model

bestModel.write().overwrite().save("models/a2_model")
```

```
print("Best model saved successfully to: models/a2_model")
```

```
Best model saved successfully to: models/a2_model
```

```
# reload saved model
```

```
loaded_model = PipelineModel.load("models/a2_model")
```

```
print("Model reloaded successfully.")
```

```
print(type(loaded_model))
```

```
Model reloaded successfully.
```

```
<class 'pyspark.ml.pipeline.PipelineModel'>
```

```
# sample prediction
```

```
sample_data = test_data.limit(5)
```

```
sample_predictions = loaded_model.transform(sample_data)
```

```
sample_predictions.select(
```

```
    "SEVERITY",
```

```
    "label",
```

```
    "prediction",
```

```
    "probability"
```

```
).show(truncate=False)
```

```
+-----+-----+-----+
+-----+-----+-----+
+-----+-----+-----+
|SEVERITY|label|prediction|probability|
+-----+-----+-----+
+-----+-----+-----+
+-----+-----+-----+
|Serious injury accident|1.0|0.0|
[0.5480584933533832,0.4410962548162602,0.010816875843241452,4.84469952
88222305E-6,2.3531287586283203E-5]|
|Serious injury accident|1.0|1.0|
[0.4228829183008683,0.5576405007894247,0.01944289259425534,5.717396690
805751E-6,2.7970918760749434E-5]|
|Serious injury accident|1.0|0.0|
[0.7418141454746979,0.2548699269919456,0.003298522674499192,2.97579767
27963994E-6,1.4429061184638107E-5]|
|Other injury accident|0.0|0.0|
[0.7801339971568104,0.21572454879882771,0.004123811181533401,3.0499230
80632834E-6,1.4592939747612884E-5]|
|Other injury accident|0.0|1.0|
[0.45092203568740447,0.533836831462489,0.015209071624685476,5.42877173
97851035E-6,2.6632453681391468E-5]|
+-----+-----+-----+
+-----+-----+-----+
```

-----+

output schema for confirmation

sample_predictions.printSchema()

root

```
|-- ACCIDENT_NO: string (nullable = true)
|-- ACCIDENT_DATE: string (nullable = true)
|-- ACCIDENT_TIME: string (nullable = true)
|-- ACCIDENT_TYPE: string (nullable = true)
|-- DAY_OF_WEEK: string (nullable = true)
|-- DCA_CODE: string (nullable = true)
|-- DCA_CODE_DESCRIPTION: string (nullable = true)
|-- LIGHT_CONDITION: string (nullable = true)
|-- POLICE_ATTEND: string (nullable = true)
|-- ROAD_GEOMETRY: string (nullable = true)
|-- SEVERITY: string (nullable = true)
|-- SPEED_ZONE: string (nullable = true)
|-- RUN_OFFROAD: string (nullable = true)
|-- ROAD_NAME: string (nullable = true)
|-- ROAD_TYPE: string (nullable = true)
|-- ROAD_ROUTE_1: string (nullable = true)
|-- LGA_NAME: string (nullable = true)
|-- DTP_REGION: string (nullable = true)
|-- LATITUDE: double (nullable = true)
|-- LONGITUDE: double (nullable = true)
|-- VICGRID_X: double (nullable = true)
|-- VICGRID_Y: double (nullable = true)
|-- TOTAL_PERSONS: integer (nullable = true)
|-- INJ_OR_FATAL: integer (nullable = true)
|-- FATALITY: integer (nullable = true)
|-- SERIOUSINJURY: integer (nullable = true)
|-- OTHERINJURY: integer (nullable = true)
|-- NONINJURED: integer (nullable = true)
|-- MALES: integer (nullable = true)
|-- FEMALES: integer (nullable = true)
|-- BICYCLIST: integer (nullable = true)
|-- PASSENGER: integer (nullable = true)
|-- DRIVER: integer (nullable = true)
|-- PEDESTRIAN: integer (nullable = true)
|-- PILLION: integer (nullable = true)
|-- MOTORCYCLIST: integer (nullable = true)
|-- UNKNOWN: integer (nullable = true)
|-- PED_CYCLIST_5_12: integer (nullable = true)
|-- PED_CYCLIST_13_18: integer (nullable = true)
|-- OLD_PED_65_AND_OVER: integer (nullable = true)
|-- OLD_DRIVER_75_AND_OVER: integer (nullable = true)
|-- YOUNG_DRIVER_18_25: integer (nullable = true)
|-- NO_OF_VEHICLES: integer (nullable = true)
```

```
-- HEAVYVEHICLE: integer (nullable = true)
-- PASSENGERVEHICLE: integer (nullable = true)
-- MOTORCYCLE: integer (nullable = true)
-- PT_VEHICLE: integer (nullable = true)
-- DEG_URBAN_NAME: string (nullable = true)
-- SRNS: string (nullable = true)
-- RMA: string (nullable = true)
-- DIVIDED: string (nullable = true)
-- STAT_DIV_NAME: string (nullable = true)
-- ACCIDENT_TYPE_index: double (nullable = false)
-- DAY_OF_WEEK_index: double (nullable = false)
-- LIGHT_CONDITION_index: double (nullable = false)
-- POLICE_ATTEND_index: double (nullable = false)
-- ROAD_GEOMETRY_index: double (nullable = false)
-- SPEED_ZONE_index: double (nullable = false)
-- RUN_OFFROAD_index: double (nullable = false)
-- DTP_REGION_index: double (nullable = false)
-- DEG_URBAN_NAME_index: double (nullable = false)
-- DIVIDED_index: double (nullable = false)
-- STAT_DIV_NAME_index: double (nullable = false)
-- ACCIDENT_TYPE_encoded: vector (nullable = true)
-- DAY_OF_WEEK_encoded: vector (nullable = true)
-- LIGHT_CONDITION_encoded: vector (nullable = true)
-- POLICE_ATTEND_encoded: vector (nullable = true)
-- ROAD_GEOMETRY_encoded: vector (nullable = true)
-- SPEED_ZONE_encoded: vector (nullable = true)
-- RUN_OFFROAD_encoded: vector (nullable = true)
-- DTP_REGION_encoded: vector (nullable = true)
-- DEG_URBAN_NAME_encoded: vector (nullable = true)
-- DIVIDED_encoded: vector (nullable = true)
-- STAT_DIV_NAME_encoded: vector (nullable = true)
-- label: double (nullable = false)
-- TOTAL_PERSONS_imputed: integer (nullable = true)
-- NO_OF_VEHICLES_imputed: integer (nullable = true)
-- MALES_imputed: integer (nullable = true)
-- FEMALES_imputed: integer (nullable = true)
-- BICYCLIST_imputed: integer (nullable = true)
-- PASSENGER_imputed: integer (nullable = true)
-- DRIVER_imputed: integer (nullable = true)
-- PEDESTRIAN_imputed: integer (nullable = true)
-- PILLION_imputed: integer (nullable = true)
-- MOTORCYCLIST_imputed: integer (nullable = true)
-- UNKNOWN_imputed: integer (nullable = true)
-- PED_CYCLIST_5_12_imputed: integer (nullable = true)
-- PED_CYCLIST_13_18_imputed: integer (nullable = true)
-- OLD_PED_65_AND_OVER_imputed: integer (nullable = true)
-- OLD_DRIVER_75_AND_OVER_imputed: integer (nullable = true)
-- HEAVYVEHICLE_imputed: integer (nullable = true)
-- YOUNG_DRIVER_18_25_imputed: integer (nullable = true)
```

```
| -- PASSENGERVEHICLE_imputed: integer (nullable = true)
| -- MOTORCYCLE_imputed: integer (nullable = true)
| -- PT_VEHICLE_imputed: integer (nullable = true)
| -- features: vector (nullable = true)
| -- rawPrediction: vector (nullable = true)
| -- probability: vector (nullable = true)
| -- prediction: double (nullable = false)
```

End to End Design Awareness

The Logistic Regression model was selected as the final model based on its performance on the test dataset. The complete pipeline is saved at `models/a2_model`

The model was reloaded and a sample operation was performed on test data to verify that the persisted artefact can be used successfully for prediction.

The model produced the expected Spark ML output, including the `label`, `features`, `rawPrediction`, `probability`, and `prediction` columns. The prediction values were also explicitly inspected to confirm that the reloaded model generates valid classification predictions.

The saved model from `models/a2_model` will be used to perform real time predictions on incoming crash records in Part B.

Part B: Streaming, Kafka and reflection

1. Streaming data simulation

2. Kafka producer

See file `producer.py`

3. Spark structured streaming consumer

4. Windowed aggregation

```
#create the spark session

from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
        .appName("VicRoadsStreamingConsumer")
        .master("local[*]")
        .getOrCreate()
)
```

26/10/04 10:57:57 WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.

#create dataframe connected to the Kafka topic

```
kafka_stream = (  
    spark.readStream  
        .format("kafka")  
        .option("kafka.bootstrap.servers", "localhost:9092")  
        .option("subscribe", "events")  
        .option("startingOffsets", "latest")  
        .load()  
)
```

#convert each message to UTF-8 string and parsed as JSON

```
json_stream = kafka_stream.select(  
    F.col("value").cast("string").alias("json")  
)
```

#explicit schema is used instead of schema inference

```
crash_schema = StructType([  
    StructField("ACCIDENT_NO", StringType(), True),  
    StructField("ACCIDENT_DATE", StringType(), True),  
    StructField("ACCIDENT_TIME", StringType(), True),  
    StructField("ACCIDENT_TYPE", StringType(), True),  
    StructField("DAY_OF_WEEK", StringType(), True),  
    StructField("DCA_CODE", StringType(), True),  
    StructField("DCA_CODE_DESCRIPTION", StringType(), True),  
    StructField("LIGHT_CONDITION", StringType(), True),  
    StructField("POLICE_ATTEND", StringType(), True),  
    StructField("ROAD_GEOMETRY", StringType(), True),  
    StructField("SEVERITY", StringType(), True),  
    StructField("SPEED_ZONE", StringType(), True),  
    StructField("RUN_OFFROAD", StringType(), True),  
    StructField("ROAD_NAME", StringType(), True),  
    StructField("ROAD_TYPE", StringType(), True),  
    StructField("ROAD_ROUTE_1", StringType(), True),  
    StructField("LGA_NAME", StringType(), True),  
    StructField("DTP_REGION", StringType(), True),  
    StructField("LATITUDE", DoubleType(), True),  
    StructField("LONGITUDE", DoubleType(), True),  
    StructField("VICGRID_X", DoubleType(), True),  
    StructField("VICGRID_Y", DoubleType(), True),  
    StructField("TOTAL_PERSONS", IntegerType(), True),  
    StructField("INJ_OR_FATAL", IntegerType(), True),  
    StructField("FATALITY", IntegerType(), True),  
    StructField("SERIOUSINJURY", IntegerType(), True),  
    StructField("OTHERINJURY", IntegerType(), True),  
    StructField("NONINJURED", IntegerType(), True),  
    StructField("MALES", IntegerType(), True),  
])
```

```

    StructField("FEMALES", IntegerType(), True),
    StructField("BICYCLIST", IntegerType(), True),
    StructField("PASSENGER", IntegerType(), True),
    StructField("DRIVER", IntegerType(), True),
    StructField("PEDESTRIAN", IntegerType(), True),
    StructField("PILLION", IntegerType(), True),
    StructField("MOTORCYCLIST", IntegerType(), True),
    StructField("UNKNOWN", IntegerType(), True),
    StructField("PED_CYCLIST_5_12", IntegerType(), True),
    StructField("PED_CYCLIST_13_18", IntegerType(), True),
    StructField("OLD_PED_65_AND_OVER", IntegerType(), True),
    StructField("OLD_DRIVER_75_AND_OVER", IntegerType(), True),
    StructField("YOUNG_DRIVER_18_25", IntegerType(), True),
    StructField("NO_OF_VEHICLES", IntegerType(), True),
    StructField("HEAVYVEHICLE", IntegerType(), True),
    StructField("PASSENGERVEHICLE", IntegerType(), True),
    StructField("MOTORCYCLE", IntegerType(), True),
    StructField("PT_VEHICLE", IntegerType(), True),
    StructField("DEG_URBAN_NAME", StringType(), True),
    StructField("SRNS", StringType(), True),
    StructField("RMA", StringType(), True),
    StructField("DIVIDED", StringType(), True),
    StructField("STAT_DIV_NAME", StringType(), True),
    StructField("event_timestamp", StringType(), True)
])

#parse each kafka message using the explicitly declared schema
#each kafka message is an array of crash records
parsed_stream = json_stream.select(
    F.from_json(
        F.col("json"),
        ArrayType(crash_schema)
    ).alias("records")
)

#explode array so each individual crash becomes one streaming dataframe
row

crash_stream = (
    parsed_stream
    .select(F.explode("records").alias("record"))
    .select("record.*")
)

#convert timestamp string into a spark timestamp
crash_stream = crash_stream.withColumn(
    "event_timestamp",
    F.to_timestamp("event_timestamp")
)

```

```

#load the model saved during part A
model = PipelineModel.load("models/a2_model")

print("Saved model loaded successfully.")

Saved model loaded successfully.

#apply the classification pipeline to each incoming streaming crash
record
predictions = model.transform(crash_stream)

```

```

#perform windowing on the predictions
windowed_predictions = (
    predictions
    .withWatermark("event_timestamp", "10 minutes")
    .groupBy(
        F.window("event_timestamp", "1 minute", "5 seconds"),
        F.col("prediction")
    )
    .count()
)

```

```

#write the windowed prediction results to the console
console_query = (
    windowed_predictions
    .writeStream
    .queryName("vicRoads_prediction_console")
    .format("console")
    .outputMode("update")
    .option("truncate", "false")
    .trigger(processingTime="5 seconds")
    .start()
)

```

```

print("Console monitoring query started.")

```

Console monitoring query started.

```

26/10/04 10:57:59 WARN ResolveWriteToStream: Temporary checkpoint
location created which is deleted normally when the query didn't fail:
/private/var/folders/9l/03xt3yvj2gg2s7lfps5hf6gh0000gn/T/temporary-
c3f5d608-2fa8-4ca1-bd2d-d480a7b777e6. If it's required to delete it
under any circumstances, please set
spark.sql.streaming.forceDeleteTempCheckpointLocation to true.
Important to know deleting temp checkpoint folder is best effort.
26/10/04 10:57:59 WARN ResolveWriteToStream:
spark.sql.adaptive.enabled is not supported in streaming
DataFrames/Datasets and will be disabled.

```

```

#write the windowed prediction results to parquet
query = (

```

```

        windowed_predictions
        .writeStream
        .queryName("vicRoads_prediction_windows")
        .format("parquet")
        .outputMode("append")
        .option("path", "output/predictions")
        .option("checkpointLocation", "checkpoints/predictions")
        .trigger(processingTime="5 seconds")
        .start()
    )

```

```
print("Streaming query started.")
```

```

26/10/04 10:57:59 WARN ResolveWriteToStream:
spark.sql.adaptive.enabled is not supported in streaming
DataFrames/Datasets and will be disabled.

```

```
Streaming query started.
```

```

26/10/04 10:57:59 WARN MicroBatchExecution: Disabling AQE since AQE is
not supported in stateful workloads.
26/10/04 10:57:59 WARN MicroBatchExecution: Disabling AQE since AQE is
not supported in stateful workloads.

```

```
-----
Batch: 0
-----
```

```

+-----+-----+-----+
|window|prediction|count|
+-----+-----+-----+
+-----+-----+-----+

```

```

print(query.isActive)
print(console_query.isActive)

```

```

True
True

```

```

#read the completed windowed prediction results from parquet
results = spark.read.parquet("output/predictions")

```

```

#display the final aggregation.
results.show(truncate=False)

```

```

query.stop()
console_query.stop()

```

Sliding Window and Watermark Justification

A 1 minute sliding window with a 5 second slide interval will monitor short term trends in predicted crash severity. The producer publishes records every 5 seconds, so a 5 second slide allows the aggregation to update at the same rate as new records arrive.

The 10 minute watermark is applied to the event time column using `withWatermark("event_timestamp", "10 minutes")`. The producer's normal batch interval is 5 seconds, so 10 minutes is substantially longer than the expected arrival interval. This provides sufficient tolerance for event time latency caused by Kafka delivery or temporary delays by Spark which accounts for late events.

The 10 minute threshold is therefore selected to accommodate delayed events. Events arriving later than the watermark threshold may no longer update completed windows.

5. Reflection section

Why was this model selected?

Logistic regression was chosen as it performed better than the Random Forest on the test set for the Vicroads crash data. You can see that the logistic regression had an accuracy of 0.6609 and F1 score of 0.6328, while the Random Forest has accuracy of 0.6425 and F1 score of 0.5452. Due to the increased accuracy and F1 score by logistic regression, this model can be seen to have a better balance between precision and recall across the predicted classes in the test set. Therefore, this was the model that was chosen to be persisted for the streaming predictions task.

Why was this window configuration selected?

A 1 minute sliding window with a 5 second sliding interval was selected for this task as the producer was developed to produce a batch of data every 5 seconds. Therefore, the sliding interval matches the batch frequency, and this allows for the aggregation to update regularly as the batches arrive and the 1 minute window provides enough observations to identify short-term changes in predicted crash severity. Overall, this allows monitoring of crash trends because you can observe changes over time rather than looking at individual events with low frequency. A 10 minute watermark was applied to allow for delayed records caused by kafka or spark processing delays that still fell into the period for late records.

What challenges arose during streaming?

It was a challenge to integrate Spark Structured Streaming with kafka. Spark could not find the kafka data source because the right connector was not available to the PySpark environment. I added the connector package but then I ran into the error of having missing kafka client classes. So I had to explicitly supply the required client and connector when launching the Jupyter notebook session. Eventually, I was able to get it running on some test data, and then extended it to execute the predictions using the saved model.

How would the solution behave with higher data volume?

There would probably be an issue with the batch processing incorporating the model transformation. This is because the current process has multiple steps to apply the model to each incoming record using a local spark session. It would be better to move away from local[*] and use a distributed spark cluster to have more resources. This would allow kafka and spark to work better in parallel, reduce processing time and help keep up with the increase in data.